Backups

Verification

Backups, and verification process use gitlab-restore pipeline to automate everything.

The pipeline fetches the ID of the latest CloudSQL backup for the GCP project it runs against (e.g. `gitlab-subscriptions-prod`). A pre-provisioned CloudSQL on `gitlab-restore` is used to restore the backup. Finally a verify job runs a set of queries to make sure that a recent database has been restored.

GitLab Analysis

The data team uses the backups as a data source to extract information without affecting the production database.

```
flowchart LR
    subgraph gitlab-restore[gitlab-restore VPC]
    customers-dot-proxy[customers-dot-proxy machine]
    restore-stgsub[restore-stgsub CloudSQL instance]
    restore-prdsub[restore-prdsub CloudSQL instance]
    end

    subgraph gitlab-analysis[gitlab-anaalysis VPC]
    extract-pipeline--query postgres-->customers-dot-proxy
    end

    gitlab-restore -.peering.-> gitlab-analysis
    gitlab-analysis -.peering.-> gitlab-restore
    customers-dot-proxy --> restore-stgsub
    customers-dot-proxy --> restore-prdsub

    style gitlab-restore fill:#88c
    style gitlab-analysis fill:#548
```

Connections from peered projects into CloudSQL instances cannot be established directly, so a proxy of sorts is needed for the `gitlab-analysis` to access the databases. A proxy VM (named `customers-dot-proxy`) is created by the `gitlab-restore` pipeline, on which a CloudSQL Auth Proxy process is running. Processes from `gitlab-analysis` connect to CloudSQL Auth Proxy as if they are connecting to a normal PostgreSQL database.

The proxy VM has a static private IP, and GCP firewall rules exists to allow connections from known subnets on `gitlab-analysis` to the `gitlab-restore` project.

The name of the PostgreSQL databases are `CustomersDot_production` and `CustomersDot_stg`, for production and staging, respectively.